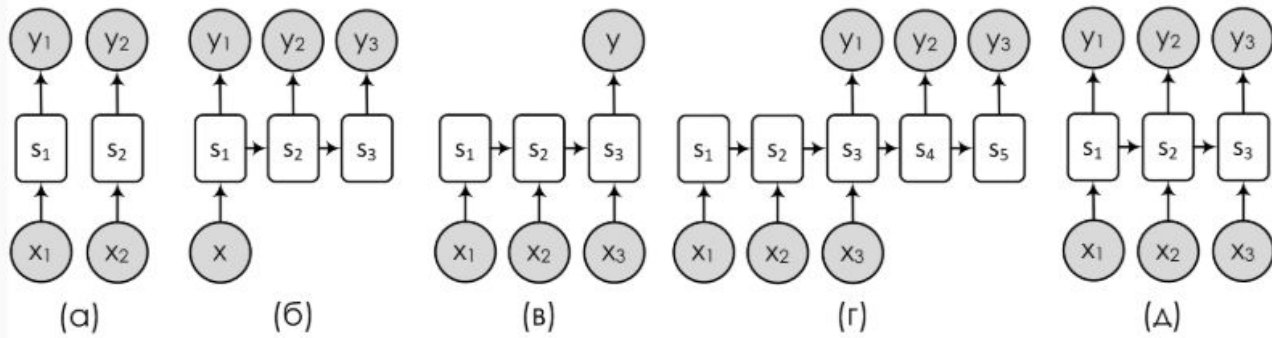


Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

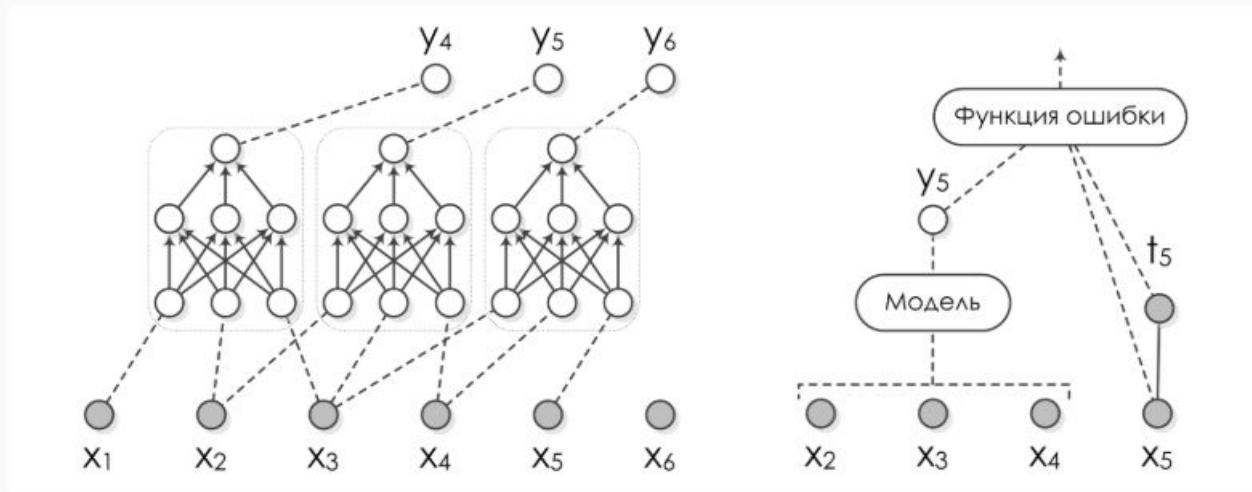
Последовательности

- Последовательности: текст, временные ряды, речь, музыка...
- Есть разные виды задач, основанных на последовательностях:



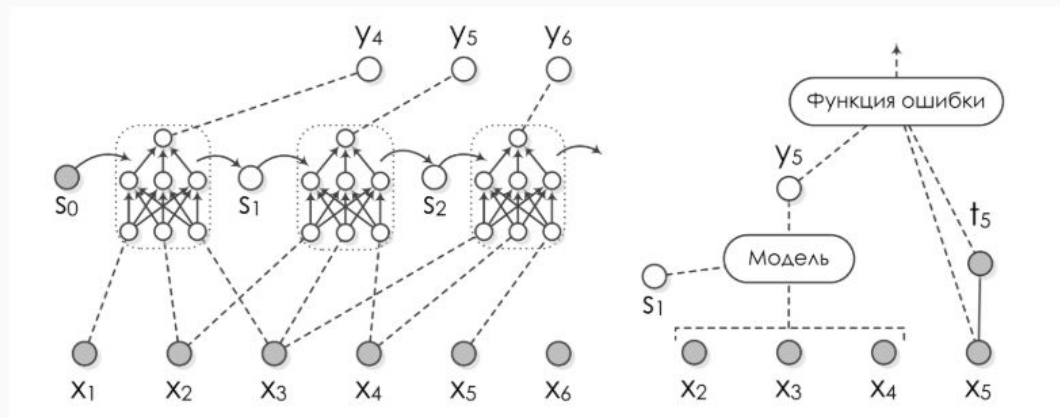
Последовательности

- Как применить к последовательности нейронную сеть?
- Можно использовать скользящее окно:



Последовательности

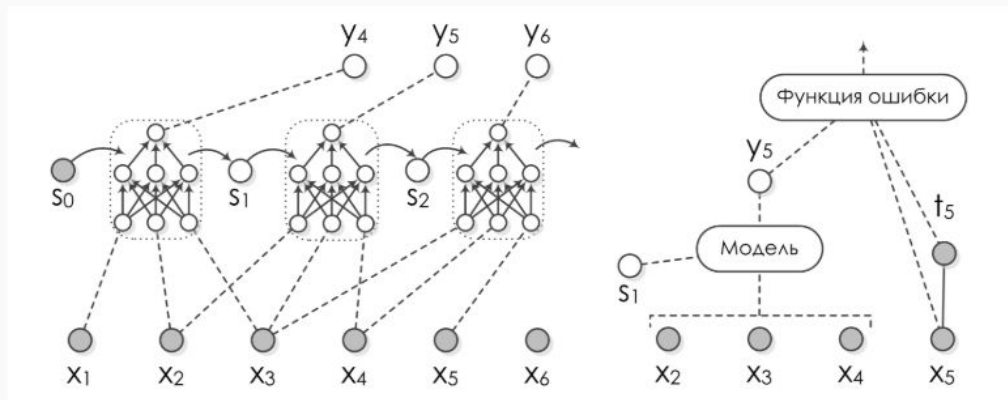
- ...но ещё лучше будет сохранять какое-нибудь скрытое состояние и обновлять его каждый раз.
- Это в точности идея *рекуррентных нейронных сетей* (recurrent neural networks, RNN).



Последовательности

- Но как теперь делать backpropagation? Получается, что в графе вычислений теперь циклы:

$$s_i = h(x_i, x_{i+1}, x_{i+2}, s_{i-1}).$$



- Это же ужасно, и всё сломалось?..

Последовательности

- ...да нет, конечно. Можно “развернуть” циклы обратно:

$$\begin{aligned}y_6 = f(x_3, x_4, x_5, s_2) &= f(x_3, x_4, x_5, h(x_2, x_3, x_4, s_1)) = \\ &= f(x_3, x_4, x_5, h(x_2, x_3, x_4, h(x_1, x_2, x_3, s_0))).\end{aligned}$$

- Так что формально проблемы нет.
- Но масса проблем в реальности: получается, что рекуррентная сеть – это такая *очень* глубокая сеть с кучей общих весов...

Идея

- Мы читаем текст последовательно
- И постепенно всё лучше понимаем, о чём он

Рекуррентные сети (RNN)

- Последовательность: $x_1, x_2, \dots, x_n, \dots$
- Читаем слева направо
- h_t — накопленная информация после чтения t элементов (вектор)

Рекуррентные сети (RNN)

- Последовательность: $x_1, x_2, \dots, x_n, \dots$
- x_i — либо one-hot вектор, либо векторное представление (word2vec, fasttext, ...)

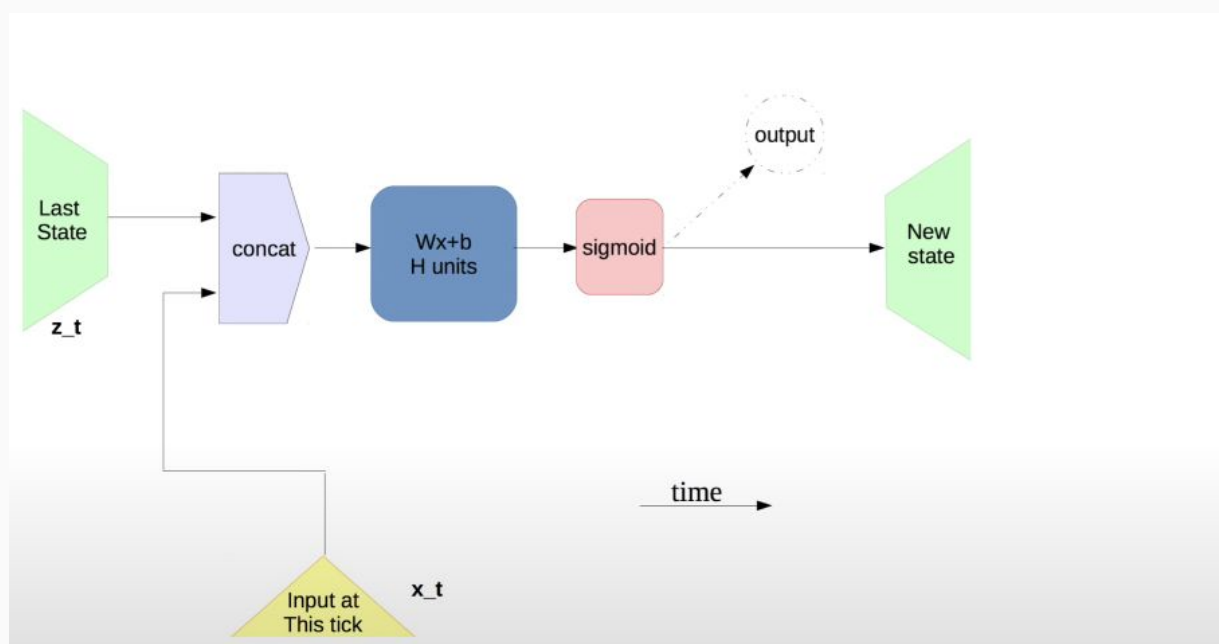
Рекуррентные сети (RNN)

- Последовательность: $x_1, x_2, \dots, x_n, \dots$
- Читаем слева направо
- h_t — накопленная информация после чтения t элементов (вектор)
- $h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$
- Если хотим что-то выдавать на каждом шаге: $o_t = f_o(W_{ho}h_t)$

Рекуррентные сети (RNN)

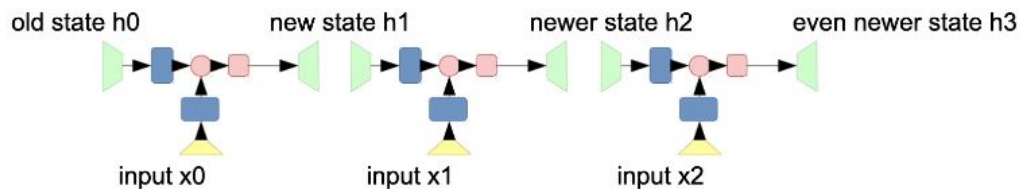
- Типичный случай: $o_t \in \mathbb{R}^N$
- N — размер словаря
- То есть предсказываем вероятность того, что здесь стоит конкретное слово
- Предсказываем следующее слово

Простая RNN



Простая RNN

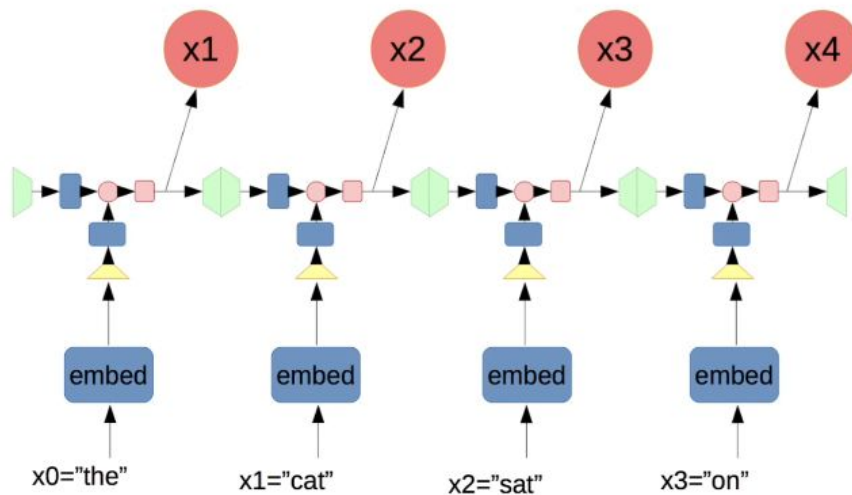
Recurrent neural network



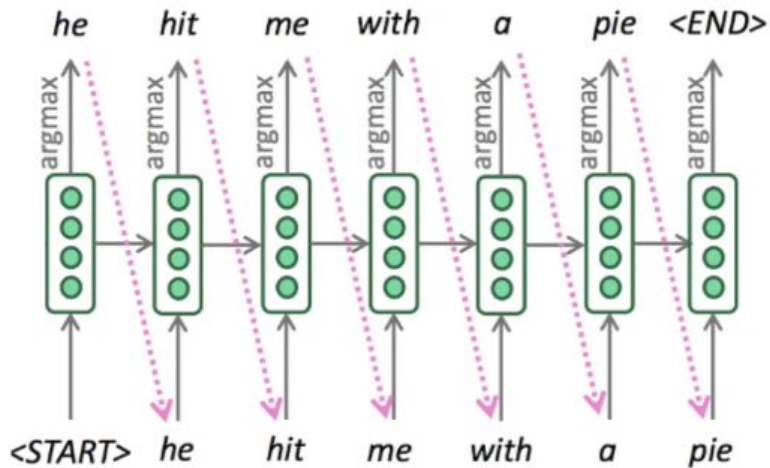
We use same weight matrices for all steps

Простая RNN

Recurrent neural network



RNN: генерация текста



RNN-блок формулами

$$h_0 = \bar{0}$$

$$h_1 = \sigma(\langle W_{\text{hid}}[h_0, x_0] \rangle + b)$$

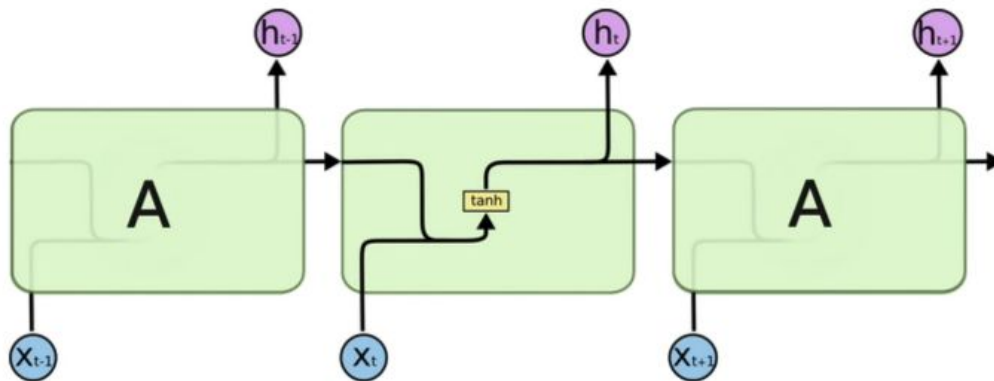
$$h_2 = \sigma(\langle W_{\text{hid}}[h_1, x_1] \rangle + b) = \sigma(\langle W_{\text{hid}}[\sigma(\langle W_{\text{hid}}[h_0, x_0] \rangle + b), x_1] \rangle + b)$$

$$h_{i+1} = \sigma(\langle W_{\text{hid}}[h_i, x_i] \rangle + b)$$

$$P(x_{i+1}) = \text{softmax}(\langle W_{\text{out}}, h_i \rangle + b_{\text{out}})$$

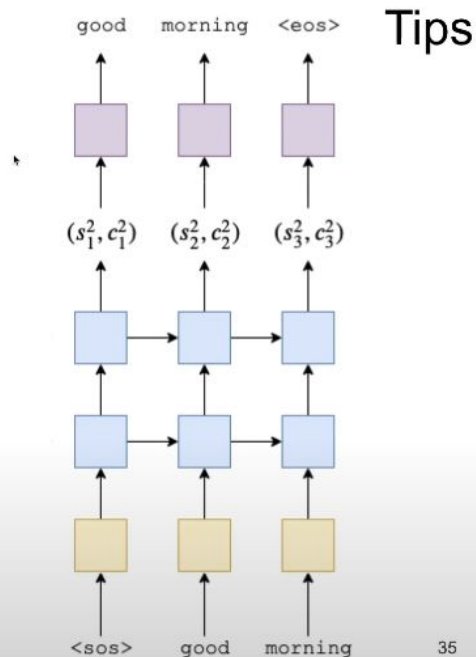
Ванильная RNN

Vanilla RNN



Зачем RNN?

- RNN is a great choice for data with sequential structure
- Multi-layer RNN can also be of great use



Градиенты RNN

- Две проблемы:
 - взрывающиеся градиенты (exploding gradients);
 - затухающие градиенты (vanishing gradients).
- Надо каждый раз умножать на одну и ту же W , и норма градиента может расти или убывать экспоненциально.
- Взрывающиеся градиенты: надо каждый раз умножать на W , и норма градиента может расти экспоненциально.
- Что делать?

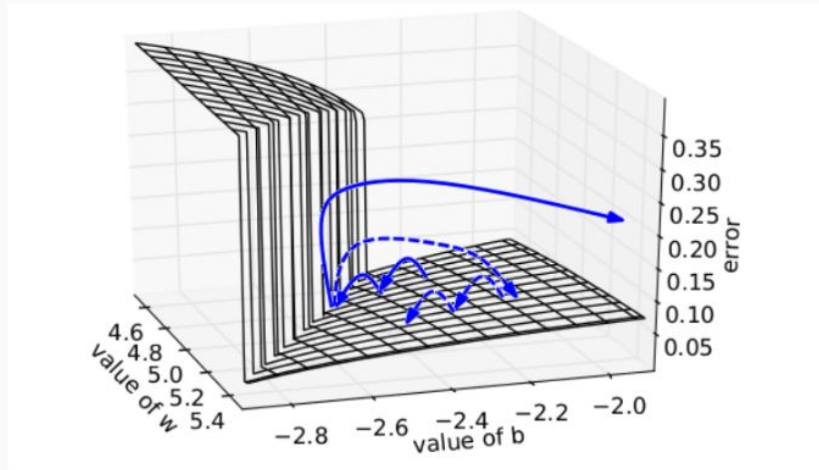
Взрывающиеся градиенты

- Да просто обрезать градиенты, ограничить сверху, чтобы не росли.

- Два варианта – ограничить общую норму или каждое значение:

- `sgd = optimizers.SGD(lr=0.01, clipnorm=1.)`
- `sgd = optimizers.SGD(lr=0.01, clipvalue=.05)`

- (Pascanu et al., 2013) – вот что будет происходить:

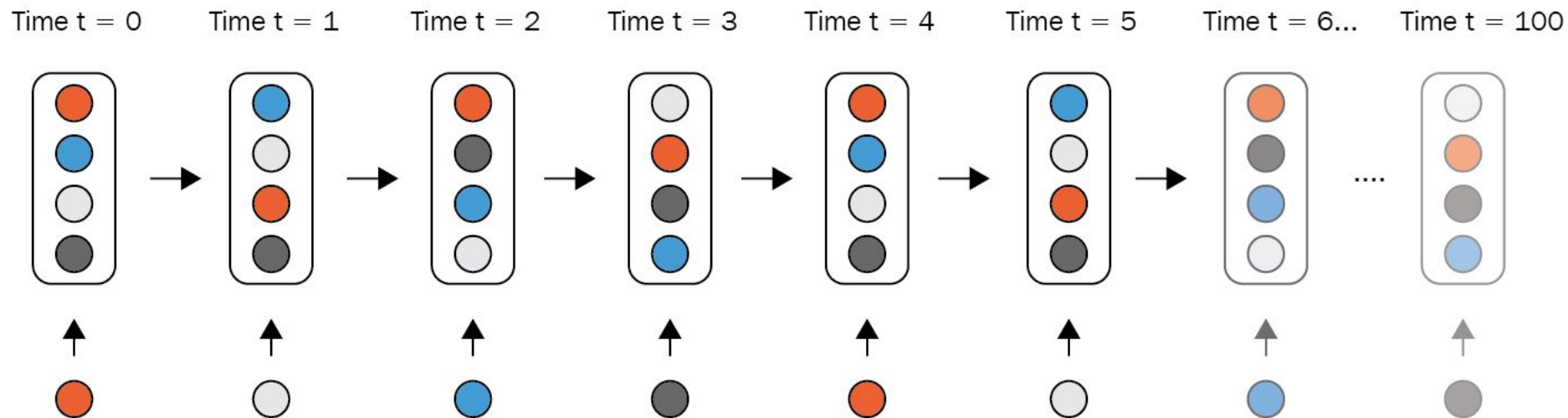


Затухающие градиенты

Проблема затухающего градиента (vanishing gradient problem) - это общая проблема, которая возникает при обучении глубоких нейронных сетей с использованием методов обратного распространения ошибки (backpropagation through time, BPTT). Эта проблема заключается в том, что при обратном распространении градиентов через множество слоев сети, градиенты могут быть очень малыми числами, что приводит к медленному обучению или замедленному обновлению весов в начальных слоях сети.

Проблема затухающего градиента особенно актуальна для рекуррентных нейронных сетей (RNN), которые используются для обработки последовательных данных, таких как текст, речь или временные ряды. В RNN градиенты распространяются через время, и чем дальше в прошлое мы идем, тем больше градиенты уменьшаются. Это означает, что информация о ранних временных шагах может быть потеряна при обратном распространении градиентов, что делает сложно обучать сеть на длинных последовательностях.

Затухающие градиенты



LSTM СЕТИ

Ячейка долгой краткосрочной памяти (Long Short- Term Memory - LSTM) была предложена в 1997 году Сеппом Хохрайтером и Юргеном Шмидхубером, а с годами понемногу совершенствовалась.

Ячейка LSTM расщепляет свое состояние на два вектора: $h(t)$ и $c(t)$ (" c " обозначает "cell"). Можно считать $h(t)$ краткосрочным состоянием, а $c(t)$ - долгосрочным состоянием.

tf.keras.layers.LSTM

[View source on GitHub](#)

Long Short-Term Memory layer - Hochreiter 1997.

Inherits From: [RNN](#), [Layer](#), [Operation](#)

[tf.keras.layers.LSTM](#) | TensorFlow v2.16.1

```
tf.keras.layers.LSTM(  
    units,  
    activation='tanh',  
    recurrent_activation='sigmoid',  
    use_bias=True,  
    kernel_initializer='glorot_uniform',  
    recurrent_initializer='orthogonal',  
    bias_initializer='zeros',  
    unit_forget_bias=True,  
    kernel_regularizer=None,  
    recurrent_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    recurrent_constraint=None,  
    bias_constraint=None,  
    dropout=0.0,  
    recurrent_dropout=0.0,  
    seed=None,  
    return_sequences=False,  
    return_state=False,  
    go_backwards=False,  
    stateful=False,  
    unroll=False,  
    use_cudnn='auto',  
    **kwargs  
)
```

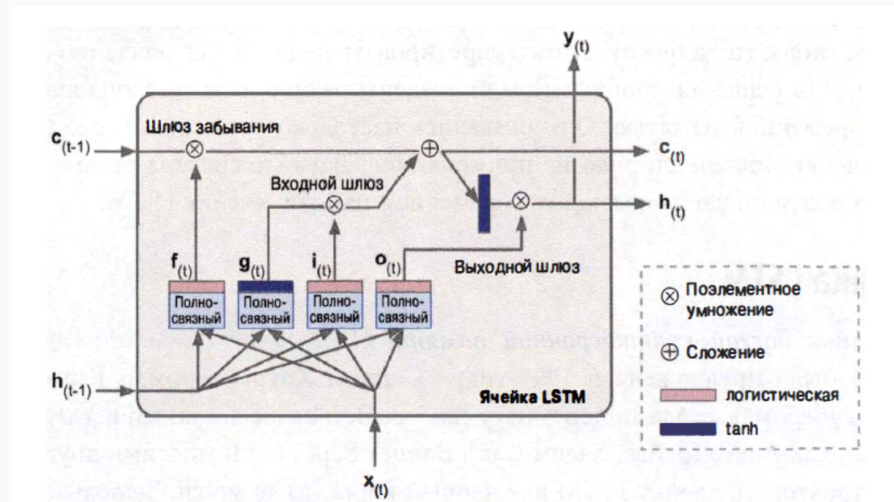

Долгая краткосрочная память (Long short-term memory; LSTM)

Во время пересечения сети слева направо долгосрочное состояние $c(t-1)$ сначала проходит через шлюз забывания (forget gate) с отбрасыванием некоторых воспоминаний и затем посредством операции сложения к нему добавляется ряд новых воспоминаний (выбранных входным шлюзом (input gate)).

Результат $c(t)$ отправляется прямо на выход без какой-либо дальнейшей трансформации. Следовательно, на каждом временном шаге одни воспоминания отбрасываются, а другие добавляются.

Кроме того, после операции сложения долгосрочное состояние копируется и пропускается через функцию $\tanh$, а результат фильтруется выходным шлюзом (output gate).

Итогом будет краткосрочное состояние $h(t)$ (которое равно выходу ячейки для данного временного шага $Y(t)$).



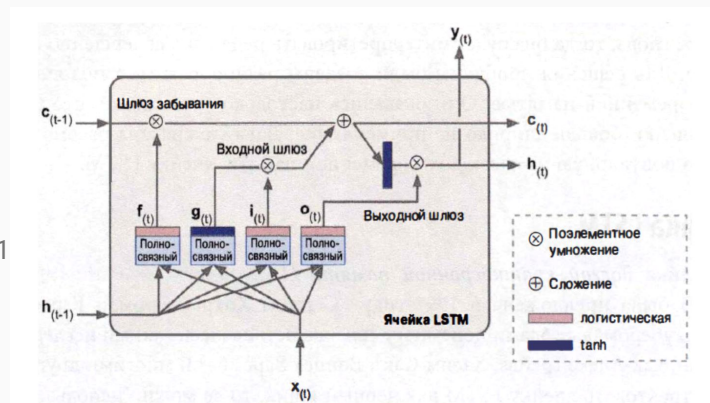
Долгая краткосрочная память (Long short-term memory; LSTM)

Первым делом текущий входной вектор $x(t)$ и предыдущее краткосрочное состояние $h(t-1)$ передаются четырем полносвязным слоям. Все они служат разным целям.

Главный слой выводит $g(t)$. Он выполняет обычную роль, анализируя текущие входы $x(t)$ и предыдущее (краткосрочное) состояние $h(t-1)$.

Остальные три слоя являются контроллерами шлюзов (gate controller). Поскольку они используют логистическую функцию активации, их выходы находятся в диапазоне от 0 до 1. Они передаются операциям поэлементного умножения, так что если они выдают нули, то закрывают шлюз, а если единицы, то открывают его. Более точно:

- шлюз забывания (контролируемый $f(t)$) управляет тем, какие части долгосрочного состояния должны быть разрушены;
- входной шлюз (контролируемый $i(t)$) управляет тем, какие части $g(t)$ должны быть добавлены к долгосрочному состоянию (именно потому речь идет только о "частичном сохранении");
- выходной шлюз (контролируемый $o(t)$) управляет тем, какие части долгосрочного состояния должны быть прочитаны и выданы на данном временном шаге (в $h(t)$ и $Y(t)$).



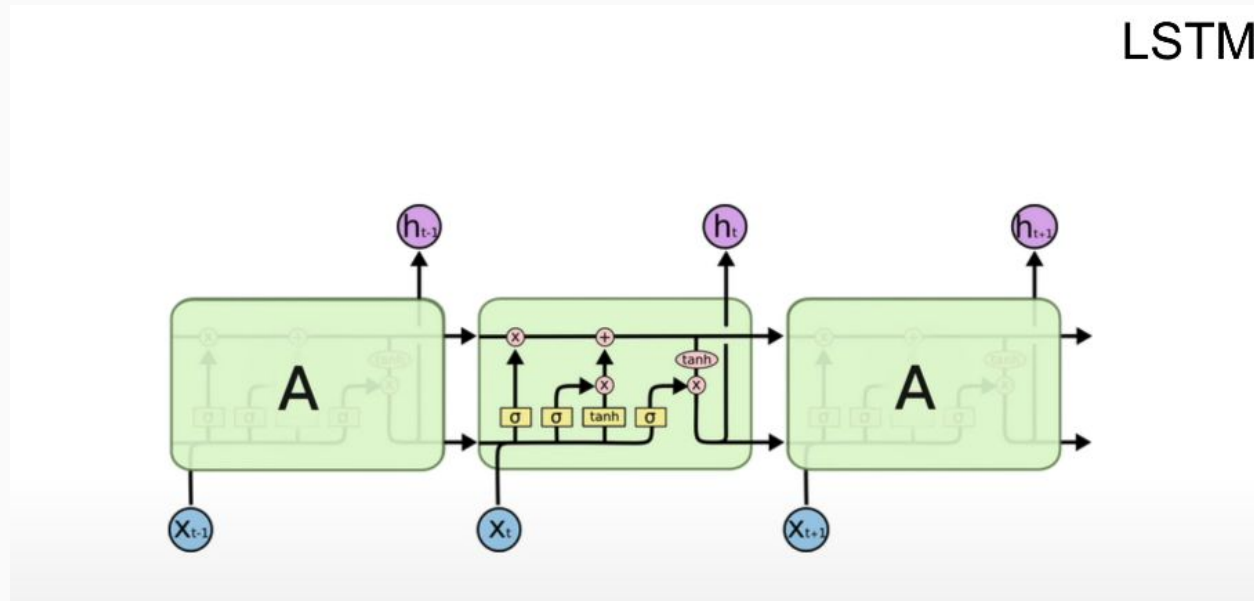
Долгая краткосрочная память (Long short-term memory; LSTM)

LSTM-блоки содержат три «вентиль», которые используются для контроля потоков информации на входах и на выходах памяти данных блоков. Эти вентили реализованы в виде логистической функции для вычисления значения в диапазоне (0; 1). Умножение на это значение используется для частичного допуска или запрещения потока информации внутрь и наружу памяти.

«Входной вентиль» контролирует меру вхождения нового значения в память, а «вентиль забывания» контролирует меру сохранения значения в памяти. «Выходной вентиль» контролирует меру того, в какой степени значение, находящееся в памяти, используется при расчёте выходной функции активации для блока. (Идея заключается в том, что старое значение следует забывать тогда, когда появится новое значение достойное запоминания).

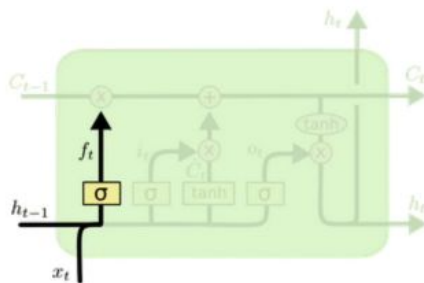
Веса в LSTM-блоке используются для задания направления оперирования вентилей. Эти веса определены для значений, которые подаются в блок (включая $x(t)$ и выход с предыдущего временного шага $h(t-1)$) для каждого из вентилей. Таким образом, LSTM-блок определяет, как распоряжаться своей памятью как функцией этих значений, и настройка весов позволяет LSTM-блоку минимизировать ошибку обучения. LSTM-блоки обычно обучают при помощи метода обратного распространения ошибки.

LSTM-Блок



LSTM-Блок 1

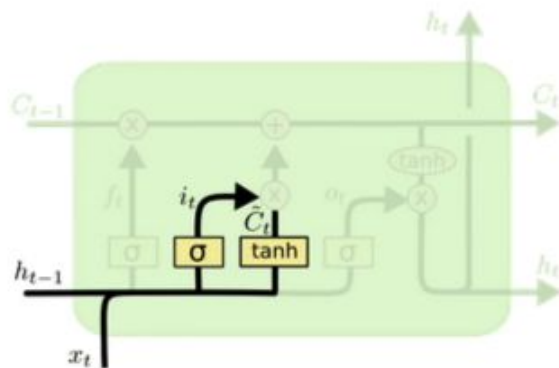
LSTM: quick overview



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTM-Блок 2

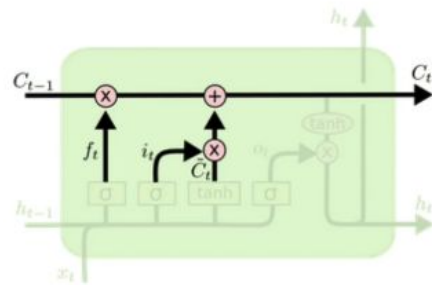
LSTM: quick overview



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM-Блок 3

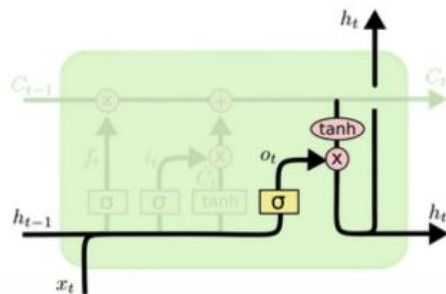
LSTM: quick overview



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM-Блок 4

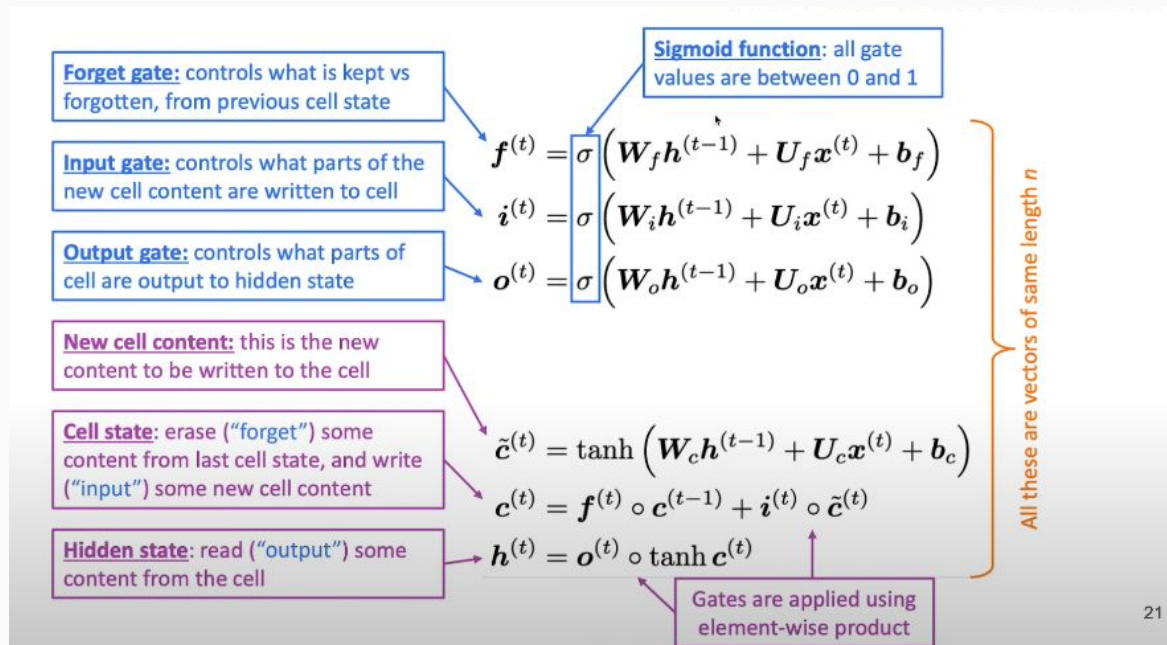
LSTM: quick overview



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

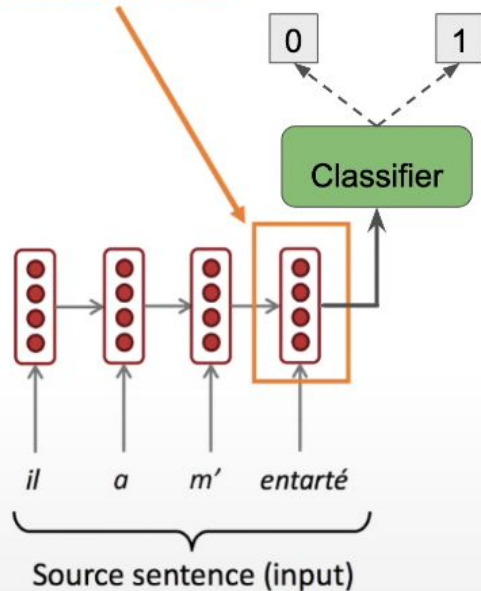
$$h_t = o_t * \tanh(C_t)$$

LSTM формулы



LSTM как кодировщик последовательности

Encoding of the
source sentence.



RNN as encoder for sequential data

RNNs can be used to encode an input sequence in a fixed size vector.

This vector can be treated as a representation of input sequence.

А что появилось после LSTM?

Ответ: Gated Recurrent Unit (GRU)

GRU и LSTM отличаются внутренней структурой и количеством взвешенных шлюзов, которые они используют для управления потоком информации через сеть.

Внутренняя структура:

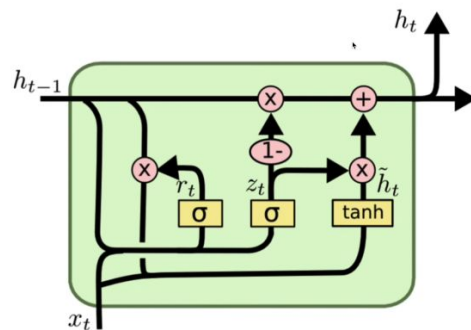
- LSTM имеет три взвешенных входных шлюза (забывающий, входной и выходной) и один взвешенный выходной шлюз.
- GRU имеет два взвешенных входных шлюза (обновление и сброс) и один взвешенный выходной шлюз.

Количество взвешенных шлюзов:

- LSTM имеет больше взвешенных шлюзов, что делает его более сложным и мощным, но также более ресурсоемким.
- GRU имеет меньше взвешенных шлюзов, что делает его более простым и быстрым в вычислениях, но менее мощным, чем LSTM.

Параметры обучения:

- LSTM имеет больше параметров обучения, что может привести к более сложному и медленному обучению.
- GRU имеет меньше параметров обучения, что может привести к более быстрому и простому обучению.



Формально GRU

Vanishing gradient: GRU

Update gate: controls what parts of hidden state are updated vs preserved

Reset gate: controls what parts of previous hidden state are used to compute new content

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$\mathbf{u}^{(t)} = \sigma \left(\mathbf{W}_u \mathbf{h}^{(t-1)} + \mathbf{U}_u \mathbf{x}^{(t)} + \mathbf{b}_u \right)$$

$$\mathbf{r}^{(t)} = \sigma \left(\mathbf{W}_r \mathbf{h}^{(t-1)} + \mathbf{U}_r \mathbf{x}^{(t)} + \mathbf{b}_r \right)$$

$$\tilde{\mathbf{h}}^{(t)} = \tanh \left(\mathbf{W}_h (\mathbf{r}^{(t)} \circ \mathbf{h}^{(t-1)}) + \mathbf{U}_h \mathbf{x}^{(t)} + \mathbf{b}_h \right)$$

$$\mathbf{h}^{(t)} = (1 - \mathbf{u}^{(t)}) \circ \mathbf{h}^{(t-1)} + \mathbf{u}^{(t)} \circ \tilde{\mathbf{h}}^{(t)}$$

How does this solve vanishing gradient?

Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

Нормализация RNN

- Вспомним batch normalization: очень хорошая штука, борется со сдвигом в переменных.
- Но применить batchnorm к RNN не получается:
 - теперь «уровень» – это шаг последовательности;
 - и получается, что веса общие, а статистики надо хранить по отдельности;
 - плохо, если последовательности разной длины;
 - совсем плохо, если при применении будет длиннее, чем в обучающей выборке.
- Что делать?

Нормализация RNN

- Нормализация по уровню (layer normalization; Ba, Kiros, Hinton, 2016): будем просто усреднять по одному уровню.
- Раньше было $a_t = W_{hh}h_{t-1} + W_{xh}x_t$.
- Теперь будет

$$h_t = f \left[\frac{g}{\sigma_t} \odot (a_t - \mu_t) + b \right],$$

$$\text{где } \mu_t = \frac{1}{H} \sum_{i=1}^H \sigma_{it}, \quad \sigma_t = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_{it} - \mu_t)^2},$$

а g и b — параметры слоя нормализации, как и раньше.

- Это может заметно улучшить результаты рекуррентной сети.

Recurrent batch normalization

- (Cooijmans et al, 2017) – всё-таки можно batchnorm в LSTM:

$$\begin{pmatrix} \tilde{\mathbf{f}}_t \\ \tilde{\mathbf{i}}_t \\ \tilde{\mathbf{o}}_t \\ \tilde{\mathbf{g}}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}$$
$$\begin{aligned} \mathbf{c}_t &= \sigma(\tilde{\mathbf{f}}_t) \odot \mathbf{c}_{t-1} + \sigma(\tilde{\mathbf{i}}_t) \odot \tanh(\tilde{\mathbf{g}}_t) \\ \mathbf{h}_t &= \sigma(\tilde{\mathbf{o}}_t) \odot \tanh(\mathbf{c}_t), \end{aligned}$$
$$\text{BN}(\mathbf{h}; \gamma, \beta) = \beta + \gamma \odot \frac{\mathbf{h} - \widehat{\mathbb{E}}[\mathbf{h}]}{\sqrt{\widehat{\text{Var}}[\mathbf{h}] + \epsilon}}$$

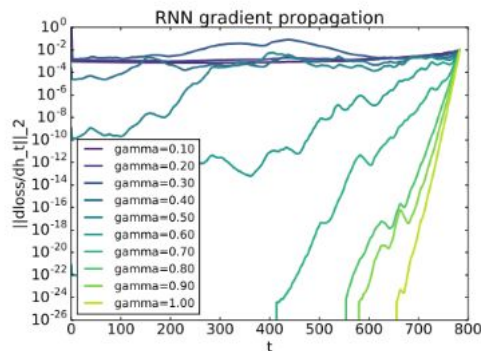
- BN для LSTM работает вот так:

$$\begin{pmatrix} \tilde{\mathbf{f}}_t \\ \tilde{\mathbf{i}}_t \\ \tilde{\mathbf{o}}_t \\ \tilde{\mathbf{g}}_t \end{pmatrix} = \text{BN}(\mathbf{W}_h \mathbf{h}_{t-1}; \gamma_h, \beta_h) + \text{BN}(\mathbf{W}_x \mathbf{x}_t; \gamma_x, \beta_x) + \mathbf{b}$$
$$\begin{aligned} \mathbf{c}_t &= \sigma(\tilde{\mathbf{f}}_t) \odot \mathbf{c}_{t-1} + \sigma(\tilde{\mathbf{i}}_t) \odot \tanh(\tilde{\mathbf{g}}_t) \\ \mathbf{h}_t &= \sigma(\tilde{\mathbf{o}}_t) \odot \tanh(\text{BN}(\mathbf{c}_t; \gamma_c, \beta_c)) \end{aligned}$$

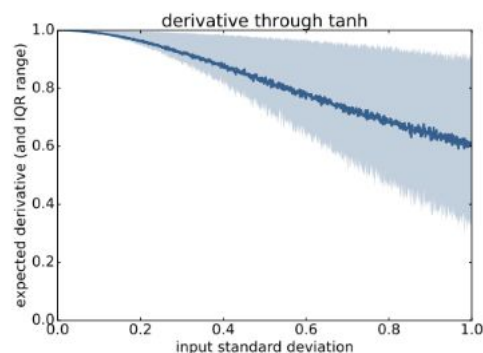
- Но где же проблемы? Почему раньше не работало?

Recurrent batch normalization

- Оказывается, параметр γ в batchnorm тут важно правильно инициализировать:



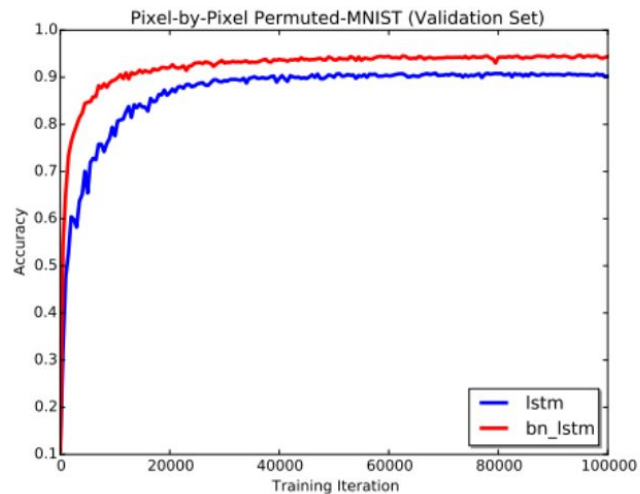
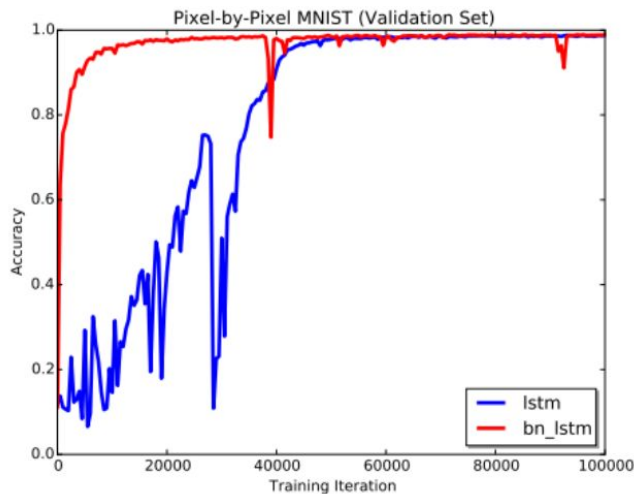
(a) We visualize the gradient flow through a batch-normalized tanh RNN as a function of γ . High variance causes vanishing gradient.



(b) We show the empirical expected derivative and interquartile range of tanh nonlinearity as a function of input variance. High variance causes saturation, which decreases the expected derivative.

Recurrent batch normalization

- И если сделать всё правильно, то получается хорошо:



Принципы работы с RNN

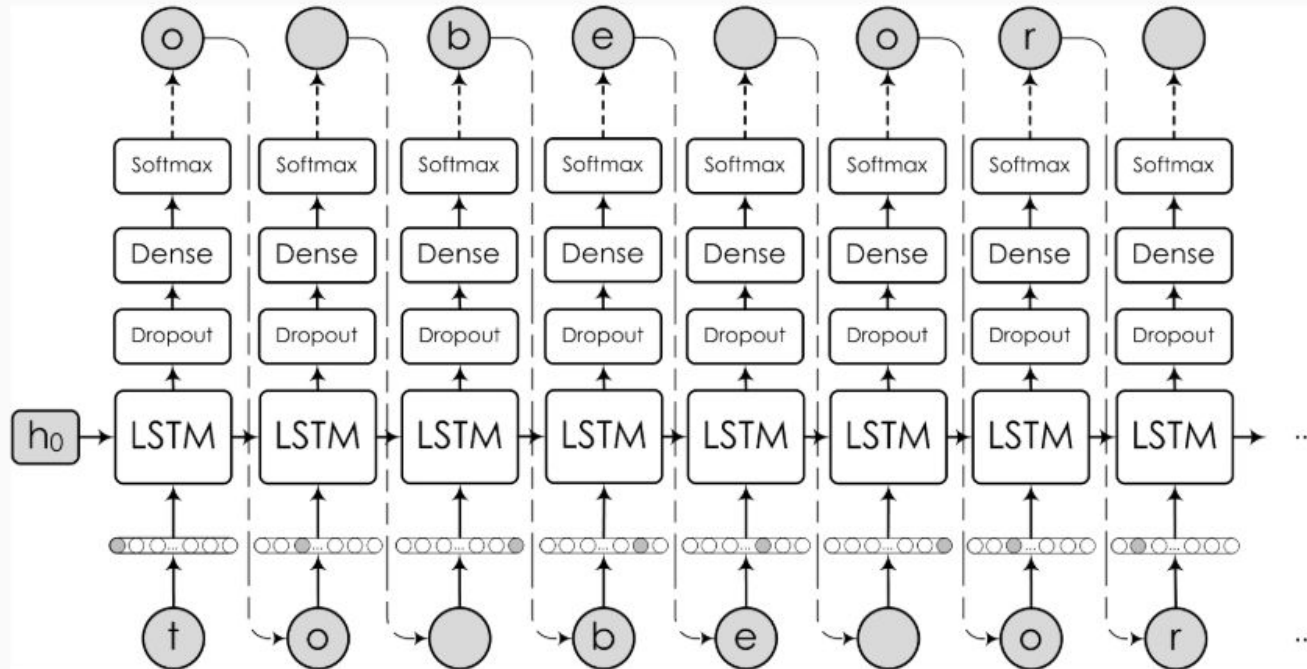
- RNN имеют довольно простую общую структуру: уровни LSTM или GRU.
- Все они выдают последовательность выходов, кроме, возможно, верхнего.
- Дропаут и batchnorm между слоями, а на рекуррентных связях надо аккуратно (потом поговорим).
- Слоёв немного; больше 3-4 трудно, 7-8 сейчас максимум.

Пример: порождение текста с RNN

- Языковые модели – это естественное прямое приложение к NLP.
- Первая идея – давайте просто обучим последовательность слов через RNN/LSTM.
- О языке будем говорить позже, а пока любопытно, что можно обучить RNN порождать интересные последовательности даже просто символ за символом.
- Karpathy, «The Unreasonable Effectiveness of Neural Networks»; знаменитый пример из (Sutskever et al. 2011):
The meaning of life is the tradition of the ancient human reproduction: it is less favorable to the good boy for when to remove her bigger...
- Это, конечно, всего лишь эффекты краткосрочной памяти, никакого «понимания».

Смысл жизни заключается в древней традиции размножения человека: хорошему мальчику менее выгодно, когда нужно убрать ее побольше.

Simple LSTM-based architecture



Немного менее простая архитектура на основе LSTM

